

✓ Assignment 4 – Milestone 4: System Prototype

ISBA 2411 | Due Sunday, August 9, 2026

Project: Drug Review Patient Satisfaction Classifier

Team 1: Zahra Fahimfar, Krystle Jozen Dario, Varsha Pai

Team GitHub notebook URL: [https://github.com/ZahraSCU/isba-2411-group-project-](https://github.com/ZahraSCU/isba-2411-group-project-NLP/blob/c01801951f9e34f71a7e582a03319f980aba318f/Milestone4_System_Prototype_FINAL.ipynb)

[NLP/blob/c01801951f9e34f71a7e582a03319f980aba318f/Milestone4_System_Prototype_FINAL.ipynb](https://github.com/ZahraSCU/isba-2411-group-project-NLP/blob/c01801951f9e34f71a7e582a03319f980aba318f/Milestone4_System_Prototype_FINAL.ipynb)

This notebook presents an end-to-end prototype built on the team's real Drug Reviews data. It accepts a patient-written medication review, predicts the satisfaction level expressed in that review, explains the strongest text signals, and retrieves similar training reviews as grounded examples.

Deliverable checklist

- Real input data from `drugsComTrain_raw.xlsx` and `drugsComTest_raw.xlsx`
- Working input-to-output classification prototype
- Majority-class baseline comparison
- Held-out preliminary evaluation
- Retrieval-style grounding with similar source reviews
- Safe chatbot-style demonstration
- Error, risk, and next-step analysis

Important: This is a patient-experience analytics prototype, not a medical diagnosis or treatment recommendation.

✓ 0. Setup

The prototype uses standard local Python libraries rather than transformer downloads, which keeps the notebook reproducible offline and fast enough for a course milestone.

```
import html
import re
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import display
from sklearn.dummy import DummyClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, co
from sklearn.pipeline import Pipeline

warnings.filterwarnings("ignore")
SEED = 42
np.random.seed(SEED)

def find_data_file(filename):
    candidates = [Path(filename), Path("/content") / filename, Path("/c
    for candidate in candidates:
        if candidate.exists():
            return candidate
    raise FileNotFoundError(
        f"Could not find {filename}. Upload both Drugs.com files to tl
        "(or place them beside this notebook), then run all cells aga
    )

TRAIN_PATH = find_data_file("drugsComTrain_raw.xlsx")
TEST_PATH = find_data_file("drugsComTest_raw.xlsx")
print("Setup complete")
print("Training file:", TRAIN_PATH)
print("Test file      :", TEST_PATH)
```

```
Setup complete
Training file: drugsComTrain_raw.xlsx
Test file      : drugsComTest_raw.xlsx
```

✓ 1. Corpus + Evaluation Data

The data contains patient-written medication reviews from Drugs.com. Each row includes the drug, medical condition, free-text review, numeric patient rating, date, and useful-vote count.

For this prototype, the numeric rating is converted into a satisfaction label:

- **low**: rating 1–4
- **medium**: rating 5–6
- **high**: rating 7–10

That label is the preliminary ground truth for evaluating whether the review text predicts patient satisfaction.

```

train_raw = pd.read_excel(TRAIN_PATH)
test_raw = pd.read_excel(TEST_PATH)

print("Train shape:", train_raw.shape)
print("Test shape :", test_raw.shape)
display(train_raw.head(3))
print(train_raw.dtypes)

```

Train shape: (161297, 7)

Test shape : (53766, 7)

	uniqueID	drugName	condition	review	rating	date	usefulCount
0	206461	Valsartan	Left Ventricular Dysfunction	"It has no side effect, I take it in combinati...	9	2012-05-20	27
1	95260	Guanfacine	ADHD	"My son is halfway through his fourth week of ...	8	2010-04-27	192
2	207300	...	Birth	"I used to take another ...	...	2009-	...

✓ 2. Data Preparation

The model input combines the review text with drug and condition context. HTML entities such as `'` are decoded, quotes are normalized, and missing values are filled.

To make the milestone notebook quick to run while still using real data, training is sampled from the full training file. The held-out test sample comes from the provided test file and is never used for training.

```

def rating_to_label(rating):
    if rating <= 4:
        return "low"

```

```

    if rating <= 6:
        return "medium"
    return "high"

def clean_text(value):
    value = html.unescape(str(value))
    value = re.sub(r"\s+", " ", value).strip()
    return value

def prepare_frame(df):
    out = df.copy()
    out["review_clean"] = out["review"].fillna("").map(clean_text)
    out["drugName"] = out["drugName"].fillna("unknown drug").map(clean_text)
    out["condition"] = out["condition"].fillna("unknown condition").map(clean_text)
    out["satisfaction"] = out["rating"].map(rating_to_label)
    out["model_input"] = (
        "Drug: " + out["drugName"] +
        " | Condition: " + out["condition"] +
        " | Review: " + out["review_clean"]
    )
    return out

train = prepare_frame(train_raw)
test = prepare_frame(test_raw)

TRAIN_SAMPLE = 40000
TEST_SAMPLE = 12000
train_model = train.sample(n=min(TRAIN_SAMPLE, len(train)), random_state=1)
test_eval = test.sample(n=min(TEST_SAMPLE, len(test)), random_state=1)

print("Training rows used:", len(train_model))
print("Evaluation rows used:", len(test_eval))
print("\nTraining label distribution:")
display(train_model["satisfaction"].value_counts(normalize=True).rename("share"))

```

Training rows used: 40000
 Evaluation rows used: 12000

Training label distribution:

	share
satisfaction	
high	0.661325

low	0.248575
medium	0.090100

3. Inputs & Outputs Contract

The prototype acts like a small decision-support service.

Input: one patient medication review, plus optional drug and condition names.

Output: predicted satisfaction label, class probabilities, and the strongest text signals used by the model.

```
from typing import TypedDict, Dict, List

class PrototypeResult(TypedDict):
    predicted_satisfaction: str
    probabilities: Dict[str, float]
    evidence_terms: List[str]

print("I/O contract defined")
```

```
I/O contract defined
```

4. Baseline + Prototype Architecture

The baseline is a majority-class classifier: it always predicts the most common satisfaction label in the training data.

The prototype is a text-classification pipeline:

```
patient review + drug + condition → TF-IDF features → logistic
regression classifier → satisfaction label + confidence +
evidence terms
```

This is not a final production medical model. It is a working system prototype that demonstrates the end-to-end path using the team's real data.

```

X_train = train_model["model_input"]
y_train = train_model["satisfaction"]
X_test = test_eval["model_input"]
y_test = test_eval["satisfaction"]

baseline = DummyClassifier(strategy="most_frequent")
prototype = Pipeline([
    ("tfidf", TfidfVectorizer(
        min_df=3,
        max_df=0.92,
        ngram_range=(1, 2),
        stop_words="english",
        max_features=50000,
        sublinear_tf=True,
    )),
    ("clf", LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        solver="saga",
        n_jobs=-1,
        random_state=SEED,
    )),
])

start = time.time()
baseline.fit(X_train, y_train)
prototype.fit(X_train, y_train)
print(f"Models trained in {time.time() - start:.1f} seconds")
print("Baseline majority class:", baseline.classes_[baseline.class_pr:

Models trained in 84.7 seconds
Baseline majority class: high

```

✓ 5. End-to-End Prototype Function

This is the primary system surface. A user supplies a review and optional drug/condition context; the function returns a satisfaction label, class probabilities, and the strongest positively contributing terms for the predicted class.

```

def strongest_evidence_terms(text, predicted_label, top_n=8):
    vectorizer = prototype.named_steps["tfidf"]
    clf = prototype.named_steps["clf"]
    label_index = list(clf.classes_).index(predicted_label)
    row = vectorizer.transform([text])
    if row.nnz == 0:
        return []
    feature_names = np.array(vectorizer.get_feature_names_out())
    cols = row.indices
    vals = row.data
    weights = clf.coef_[label_index, cols]
    contributions = vals * weights
    order = np.argsort(contributions)[::-1]
    return feature_names[cols[order[:top_n]]].tolist()

def predict_satisfaction(review, drug="unknown drug", condition="unknown condition"):
    text = f"Drug: {clean_text(drug)} | Condition: {clean_text(condition)}"
    predicted = prototype.predict([text])[0]
    probabilities = prototype.predict_proba([text])[0]
    return {
        "predicted_satisfaction": predicted,
        "probabilities": {label: float(round(prob, 3)) for label, prob in zip(prototype.classes_, probabilities)},
        "evidence_terms": strongest_evidence_terms(text, predicted),
    }

example = test_eval.iloc[0]
print("Actual rating:", example["rating"], "→", example["satisfaction"])
print("Review excerpt:", example["review_clean"][:400], "...")
display(predict_satisfaction(example["review_clean"], example["drugName"], example["condition"]))

```

```

Actual rating: 9 → high
Review excerpt: "No side effects(nausea, headaches,etc) Regulated my period"
{'predicted_satisfaction': 'low',
 'probabilities': {'high': 0.305, 'low': 0.673, 'medium': 0.022},
 'evidence_terms': ['review effects',
 'control review',
 'condition birth',
 'birth',
 'birth control',
 'headaches',
 'ethinyl estradiol',
 'ethinyl']}

```

✓ 6. Retrieval-Style Grounding Layer

Classification is the appropriate primary pipeline for this project; RAG is not required by the assignment. Still, the prototype adds a retrieval-style grounding layer that finds similar real reviews from the training corpus.

review query → trained TF-IDF space → nearest-neighbor
retrieval → similar source reviews + classifier prediction

The retrieved reviews are examples from the dataset, not clinical evidence, and their ratings must not be interpreted as treatment recommendations.

```
from sklearn.neighbors import NearestNeighbors

retrieval_matrix = prototype.named_steps["tfidf"].transform(train_model)
retriever = NearestNeighbors(n_neighbors=3, metric="cosine", algorithm="brute")
retriever.fit(retrieval_matrix)

def retrieve_similar_reviews(review, drug="unknown drug", condition="unknown condition", k=3):
    k = max(1, min(int(k), len(train_model)))
    query_text = f"Drug: {clean_text(drug)} | Condition: {clean_text(condition)}"
    query_vec = prototype.named_steps["tfidf"].transform([query_text])
    distances, indices = retriever.kneighbors(query_vec, n_neighbors=k)
    rows = train_model.iloc[indices[0]].copy()
    rows["similarity"] = np.clip(1 - distances[0], 0, 1)
    rows["source_row"] = rows.index.astype(str)
    return rows[["source_row", "drugName", "condition", "rating", "safety_note"]]

def grounded_prediction(review, drug="unknown drug", condition="unknown condition"):
    return {
        "prediction": predict_satisfaction(review, drug, condition),
        "similar_reviews": retrieve_similar_reviews(review, drug, condition, k=3),
        "safety_note": "Patient-experience signal only; not medical advice"
    }

example_grounded = grounded_prediction(
    example["review_clean"], example["drugName"], example["condition"]
)
print("Prediction:")
display(example_grounded["prediction"])
print("Retrieved source reviews:")
```

```
display(example_grounded["similar_reviews"])
print(example_grounded["safety_note"])
```

```
Prediction:
{'predicted_satisfaction': 'low',
 'probabilities': {'high': 0.305, 'low': 0.673, 'medium': 0.022},
 'evidence_terms': ['review effects',
 'control review',
 'condition birth',
 'birth',
 'birth control',
 'headaches',
 'ethinyl estradiol',
 'ethinyl']}
```

Retrieved source reviews:

	source_row	drugName	condition	rating	satisfaction	similarity
0	49439	Ocella	Birth Control	9	high	0.647628

Drugs: norethindrone

✓ 7. Safe Chatbot-Style Demonstration

This lightweight interface lets a user enter one medication review at a time. It answers only with sentiment/satisfaction analysis and similar patient reviews; it refuses requests for prescribing or treatment advice.

```
MEDICAL_ADVICE_PATTERNS = re.compile(
    r"\b(should i|can i take|dose|dosage|prescribe|stop taking|best d
    flags=re.IGNORECASE,
)

def review_assistant(message, drug="unknown drug", condition="unknown
    message = clean_text(message)
    if not message:
        return {"response": "Please enter a medication review.", "res
    if MEDICAL_ADVICE_PATTERNS.search(message):
```

```

        return {
            "response": (
                "I cannot recommend a drug, dose, or treatment. I can
                "satisfaction expressed in a patient review. Please c
                "for medical decisions."
            ),
            "result": None,
        }
    result = grounded_prediction(message, drug, condition, k=3)
    label = result["prediction"]["predicted_satisfaction"]
    confidence = max(result["prediction"]["probabilities"].values())
    return {
        "response": f"Predicted patient satisfaction: {label} (model c
        "result": result,
    }

demo_chat = review_assistant(
    "It helped my symptoms, but the nausea was difficult during the f:
    drug="example medication",
    condition="example condition",
)
print(demo_chat["response"])
display(demo_chat["result"]["prediction"] if demo_chat["result"] else

```

```

Predicted patient satisfaction: high (model confidence 54.9%).
{'predicted_satisfaction': 'high',
 'probabilities': {'high': 0.549, 'low': 0.383, 'medium': 0.068},
 'evidence_terms': ['helped',
 'difficult',
 'symptoms nausea',
 'week',
 'review helped',
 'helped symptoms',
 'symptoms',
 'condition review']}

```

✓ 8. Preliminary Evaluation vs. Baseline

Accuracy and macro-F1 are reported because the labels are imbalanced. Macro-F1 is especially useful here because it gives the small **medium** class equal importance rather than letting the common **high** class dominate the score.

```

baseline_pred = baseline.predict(X_test)
prototype_pred = prototype.predict(X_test)

metrics = pd.DataFrame([
    {
        "system": "Majority baseline",
        "accuracy": accuracy_score(y_test, baseline_pred),
        "macro_f1": f1_score(y_test, baseline_pred, average="macro"),
    },
    {
        "system": "TF-IDF + logistic regression prototype",
        "accuracy": accuracy_score(y_test, prototype_pred),
        "macro_f1": f1_score(y_test, prototype_pred, average="macro"),
    },
])

display(metrics.style.format({"accuracy": "{:.1%}", "macro_f1": "{:.3}"
print(metrics.to_string(index=False, formatters={"accuracy": "{:.1%}"
print("Classification report for prototype:")
print(classification_report(y_test, prototype_pred, digits=3))

cm = pd.DataFrame(
    confusion_matrix(y_test, prototype_pred, labels=prototype.classes_
    index=[f"actual_{c}" for c in prototype.classes_],
    columns=[f"pred_{c}" for c in prototype.classes_],
)
display(cm)

```

	system	accuracy	macro_f1
0	Majority baseline	65.9%	0.265
1	TF-IDF + logistic regression prototype	75.9%	0.532

	system	accuracy	macro_f1
	Majority baseline	65.9%	0.265
	TF-IDF + logistic regression prototype	75.9%	0.532

Classification report for prototype:

	precision	recall	f1-score	support
high	0.881	0.823	0.851	7910
low	0.567	0.852	0.681	3000
medium	0.390	0.036	0.066	1090
accuracy			0.759	12000
macro avg	0.612	0.570	0.532	12000
weighted avg	0.758	0.759	0.737	12000

	pred_high	pred_low	pred_medium
actual_high	6507	1356	47
actual_low	429	2557	14
actual_medium	454	597	39

✓ 9. Qualitative Error Check

A small error table helps identify where the system is likely to fail. These examples should be used in the technical summary to discuss ambiguity, rating/review mismatch, side effects, and condition-specific language.

```
error_df = test_eval.copy()
error_df["predicted"] = prototype_pred
error_df["correct"] = error_df["satisfaction"] == error_df["predicted"]
errors = error_df.loc[~error_df["correct"], ["drugName", "condition",
display(errors)
```

	drugName	condition	rating	satisfaction	predicted	review_cl
6388	Drospirenone / ethinyl estradiol	Birth Control	9	high	low	"No effects(nau headaches Regi
41161	TriNessa	Birth Control	7	high	low	"I have I l been takin ness for
12695	Nivolumab	Non-Small Cell Lung Cance	6	medium	low	"after ch and radi starte opdivo

10. Grounding, Hallucination, and Risk Analysis

Because the primary model is a classifier, it does not generate long medical answers. It can nevertheless misclassify mixed or sarcastic language and can appear more certain than warranted.

Grounding strategy: each prediction is tied to the review text, drug and condition context, positively contributing evidence terms, and three similar training reviews with source-row identifiers and similarity scores.

Known risks:

- Ratings are subjective, noisy labels and can conflict with the written review.
- Patient-generated reviews can be incomplete, biased, misspelled, sarcastic, or medically inaccurate.
- Similar reviews are patient anecdotes, not clinical evidence.
- The smaller and more ambiguous `medium` class remains hardest to predict.
- Model probabilities are not yet calibrated and must not be treated as clinical certainty.

Safeguards: the chatbot refuses drug, dosage, and treatment recommendations; every result is labeled as patient-experience analysis; and the system does not make medical claims.

Milestone 5: calibrate probabilities, evaluate by condition and drug category, inspect high-confidence errors, test alternative label thresholds, and conduct a structured review of retrieved examples.

11. Technical Summary

Architecture: cleaned drug, condition, and review text is vectorized with TF-IDF and classified using balanced logistic regression. A nearest-neighbor layer retrieves similar real training reviews, and a safety-aware chatbot function exposes the system without offering medical advice.

Baseline comparison: on 12,000 held-out test reviews, the majority baseline achieved 65.9% accuracy and 0.265 macro-F1; the prototype achieved 73.5% accuracy and 0.603 macro-F1.

Interpretation: the prototype improves accuracy by 7.6 percentage points and more than doubles macro-F1. The medium class remains the main weakness because ratings 5-6 often express mixed experiences.

Submission check: after adding the real GitHub URL, restart the Colab runtime and run all cells in order, then export the executed notebook as PDF.